



University
of Exeter

COMM117

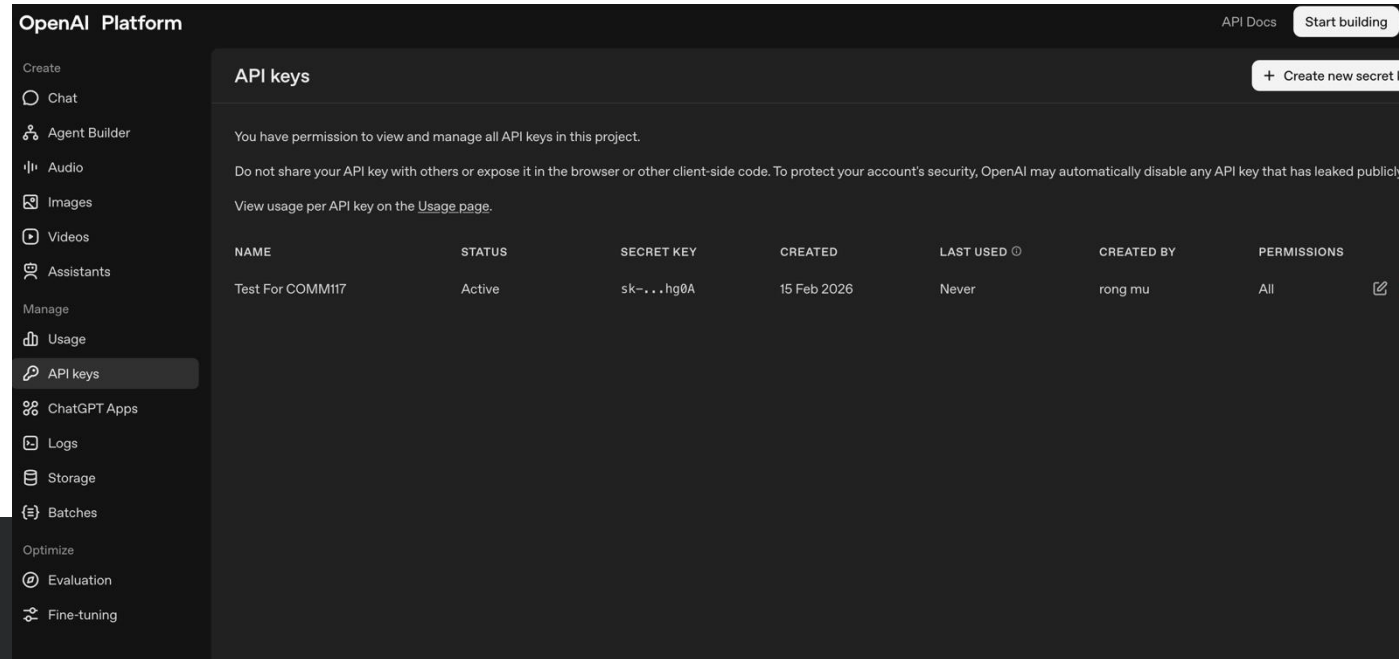
Large Language Models and Application

APIs, Frameworks, Practical Tools

How we “use” models today?

- Hosted API (OpenAI-style)
- Self-host open weights (Hugging Face / vLLM)

LLM API -- ChatGPT



```
import os
from openai import OpenAI

client = OpenAI(api_key=OPENAI_API_KEY)

resp = client.responses.create(
    model="gpt-5.2",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(resp.output_text)
```

LLM API-- Keys

```
▶ from google import genai

import os

os.environ["GEMINI_API_KEY"] = API_KEY
# The client gets the API key from the environment variable `GEMINI_API_KEY`.
client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview", contents="Explain how AI works in a few words"
)
print(response.text)

... AI analyzes data to identify patterns and make predictions.
```

Free!

<https://ai.google.dev/gemini-api/docs>

Hugging Face: core building blocks



```
import os
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, get_linear_schedule_with_warmup
from torch.utils.data import DataLoader
from accelerate import Accelerator
from peft import LoraConfig, get_peft_model, TaskType
```

```
model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # GPT2 has no pad by default
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)

lora_cfg = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["c_attn", "c_proj"], # GPT2 attention/projection modules
)
```

Transformer

LORA

Hugging Face generation parameters



When an LLM generates text, at each step it chooses the **next token**. Hugging Face lets you control **how it chooses**.

- **Length:** `max_new_tokens`: maximum tokens to generate

- **Determinism vs creativity:**

Greedy decoding (most deterministic): no sampling, always pick highest probability token `do_sample=False`

Sampling (more varied): choose probabilistically:

`do_sample=True`

`temperature`: lower = more conservative, higher = more random

`top_p` (nucleus sampling): sample from smallest set of tokens whose total prob $\geq p$

`top_k`: sample from top k most likely tokens

- **Repetition control:**

Discourages repeating tokens `repetition_penalty`

Prevents repeating n-grams (e.g., phrases) `no_repeat_ngram_size`

Takeaway: decoding settings change *style/variance*, not the model's knowledge.

vLLM



Efficient Memory Management for Large Language Model Serving with PagedAttention

- vLLM is an **open-source library** designed for fast, easy-to-use, and cost-efficient **LLM inference and serving**
- vLLM includes both **an inference server** (which manages network traffic), and **an inference engine** (to maximize computational speed).

<https://github.com/vllm-project/vllm?tab=readme-ov-file>

KV cache: why it matters

KV cache (Key–Value cache) is a **memory store** used during Transformer **inference**.

In self-attention, each layer creates **Q, K, V** (Query, Key, Value) vectors.

- During generation, the **new token** produces a new **Q**.
- The model needs **K and V for all previous tokens** to compute attention.
- So **we cache K and V** from earlier tokens and reuse them.

Without KV cache:

- For token 1...T, every new token would redo attention over all past tokens → **slow**.

With KV cache:

- **Past K/V are reused** → each step only computes K/V for the new token.
- **Big speedup**, especially for long contexts and multi-turn chat. **More GPU memory use!**

Batching strategies

1. Static batching



Good for: offline jobs / predictable traffic
Downside: increases user waiting time (latency)

2. Dynamic batching (most common in production)



Good for: real-time apps with variable traffic
Downside: needs careful limits (otherwise tail latency explodes)

3. Continuous batching / token-level batching (what vLLM is known for)

- During decoding steps.
- The server keeps across many

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

t generation
ly now

Good for: many concurrent chats
Downside: more complex scheduling, but much higher throughput

Introduction Today

1. Paper discussion: vLLM
2. Text Generation: beam search, sampling,
top-k nucleus sampling
3. Speculative decoding

vLLM paper (PagedAttention)

Motivation:

The serving bottleneck

In serving, **memory** (KV cache) is often the limiter, not FLOPs.

- Many concurrent users → long contexts → KV cache grows
- Memory fragmentation + allocation overhead hurts throughput

Abstract

High throughput serving of large language models (LLMs) requires batching sufficiently many requests at a time. However, existing systems struggle because **the key-value cache (KV cache) memory for each request is huge and grows and shrinks dynamically**. When managed inefficiently, this memory can be significantly wasted by fragmentation and redundant duplication, limiting the batch size. To address this problem, we propose **PagedAttention**, an attention algorithm inspired by the classical virtual memory and paging techniques in operating systems. On top of it, we **build vLLM**, an LLM serving system that achieves (1) near-zero waste in KV cache memory and (2) flexible sharing of KV cache within and across requests to further reduce memory usage. Our evaluations show that **vLLM improves the throughput of popular LLMs** by 2-4× with the same level of latency compared to the state-of-the-art systems, such as FasterTransformer and Orca. The improvement is more pronounced with longer sequences, larger models, and more complex decoding algorithms. vLLM's source code is publicly available at <https://github.com/vllm-project/vllm>.

vLLM paper (PagedAttention)

Motivation:

The serving bottleneck

In serving, **memory** (KV cache) is often the limiter, not FLOPs.

- Many concurrent users → long contexts → KV cache grows
- Memory fragmentation + allocation overhead hurts throughput

Why traditional KV cache wastes memory?

- Requests have **different sequence lengths**
- **GPU memory** ends up with **holes** (fragmentation)
- Result: **fewer sequences can fit** → **lower throughput**

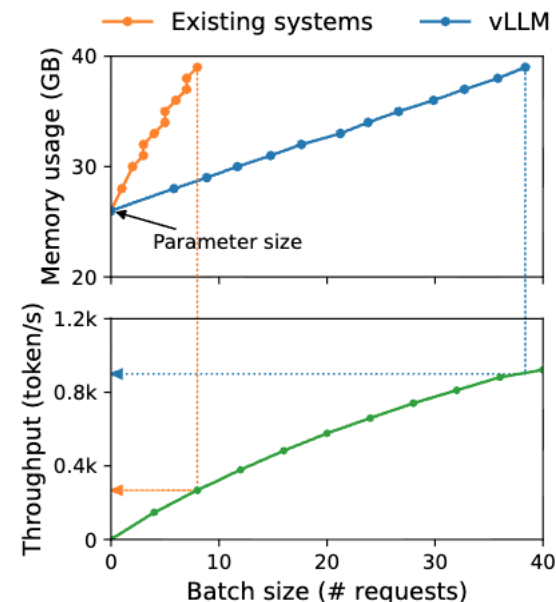
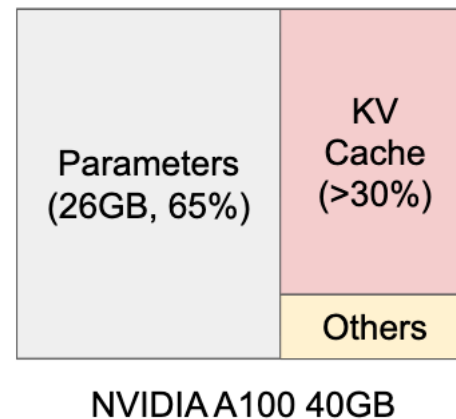
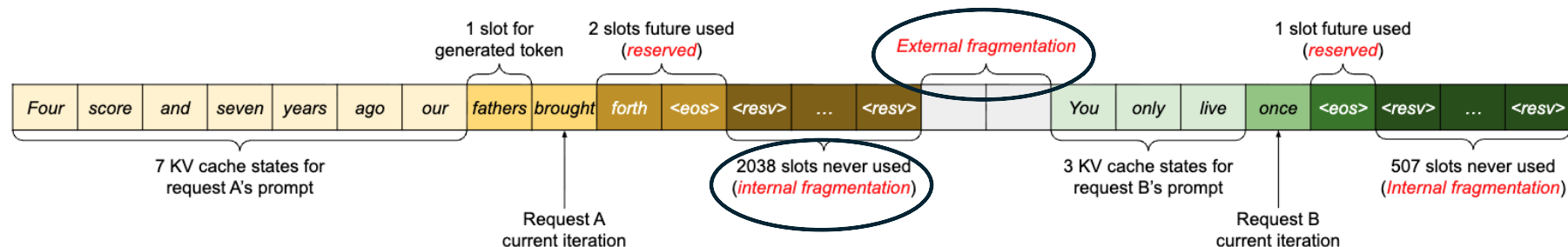
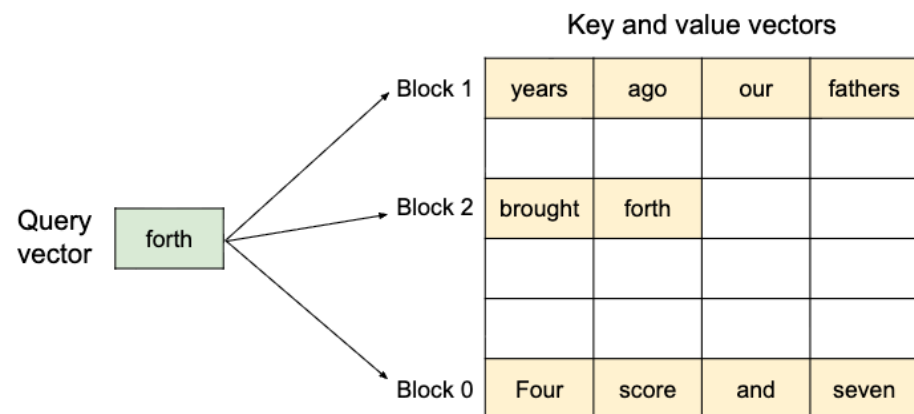


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemerally for activation. *Right:* vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems [31, 60], leading to a notable boost in serving throughput.

vLLM idea



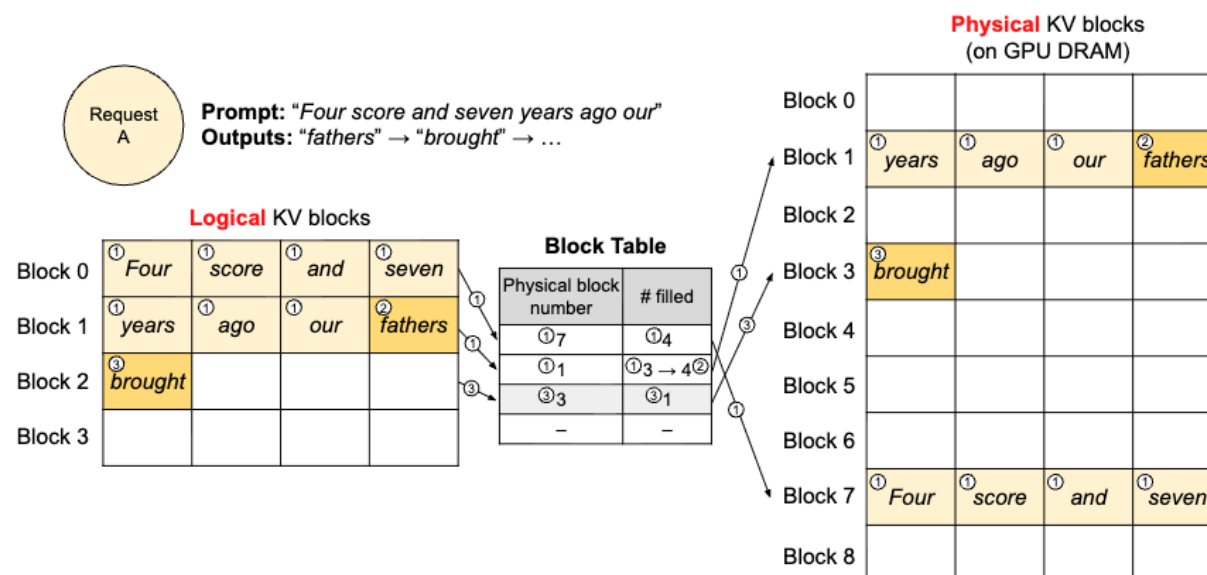
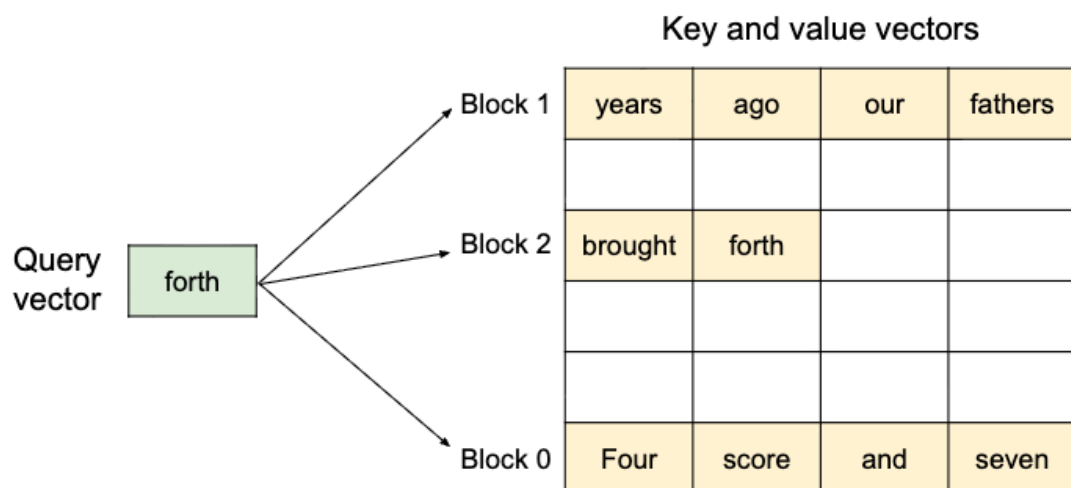
1. **Internal fragmentation** (Reserved but unused space) : Servers reserve extra KV space “just in case” a sequence gets long. If it doesn’t, that reserved memory is wasted.
2. **External fragmentation** (holes between allocations): Requests finish at different times. When one frees memory, the later needs a large chunk.



vLLM idea

PagedAttention: manage KV cache like **virtual memory paging**

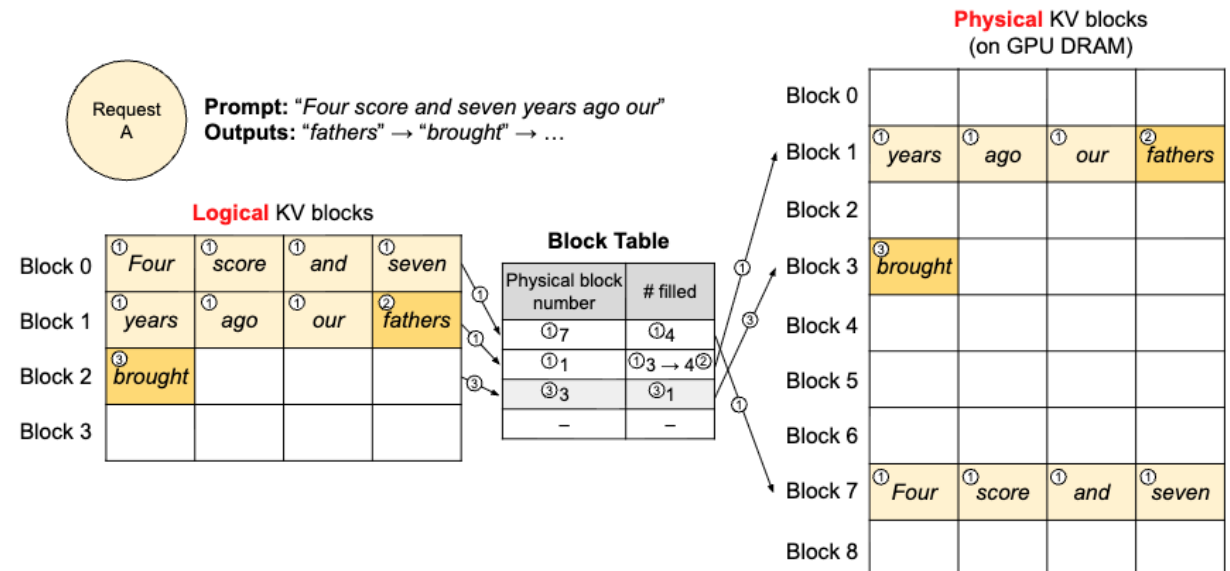
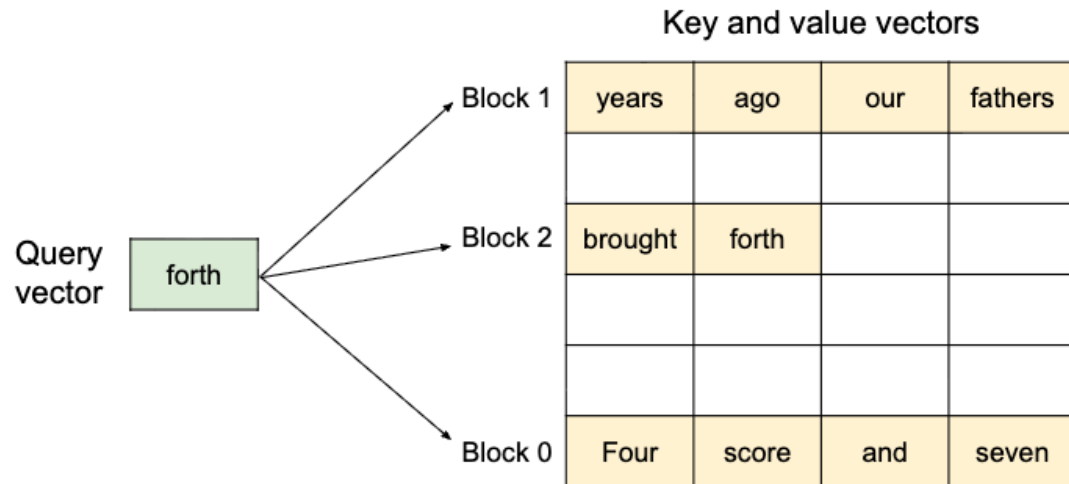
- Instead of “one big contiguous KV array per request,” KV is stored as **fixed-size blocks (pages)**.
- Each sequence has a **logical sequence of blocks**, but those blocks can live **anywhere** in GPU memory (non-contiguous).
- The system keeps a **block table** (a mapping) from:
“sequence S, block i” → “physical memory block address”



vLLM idea – PagedAttention

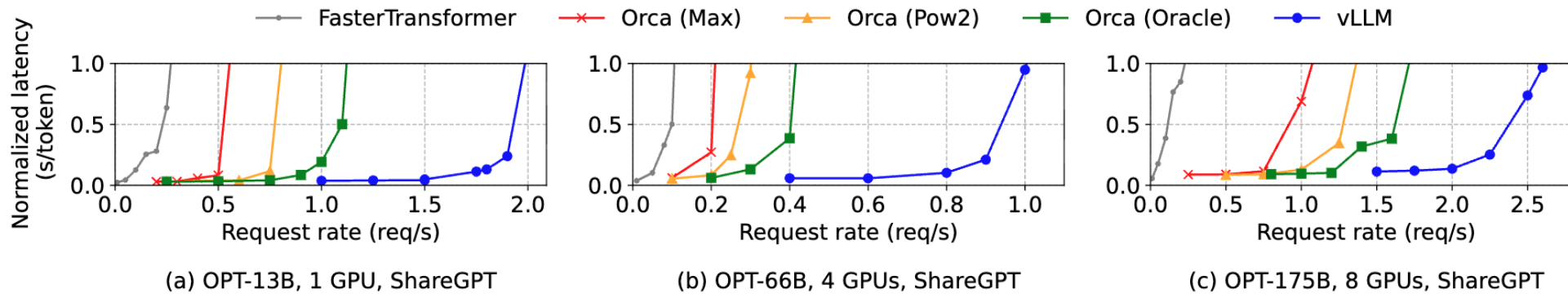
Why this improves throughput

- More concurrent requests fit in memory
- Less wasted memory → better batching
- Better scheduling for continuous batching



vLLM idea – take away

vLLM increases serving throughput mainly by **making KV cache memory management efficient** (PagedAttention), so you can **serve more sequences per GPU**.



vLLM sustains **higher request rates at similar latency** because PagedAttention reduces KV-cache waste and enables larger effective batching.

Remind: decoding is the bottleneck

In chat serving, most time is spent in **decode** (token-by-token generation).

Two constraints dominate:

- **Memory** (KV cache limits concurrency)
- **Compute** (one forward pass per token)

Paged Attention

- **Keeps KV cache compact** (less fragmentation)
- Enables **higher concurrency** and better **continuous batching**

?

- An LLM generates its **output one token at a time**, where a token is usually a word or a part of a word.
“One small step for man, one giant leap for mankind” 12 tokens → run the decoding step 12 times
- The **larger** the LLM, the more **complex** it can be. However, these larger models are also **slower**

Text Generation

Decoding = “how we pick the next token”

At each step the model gives a probability distribution over tokens.
Decoding decides **which token to choose**.

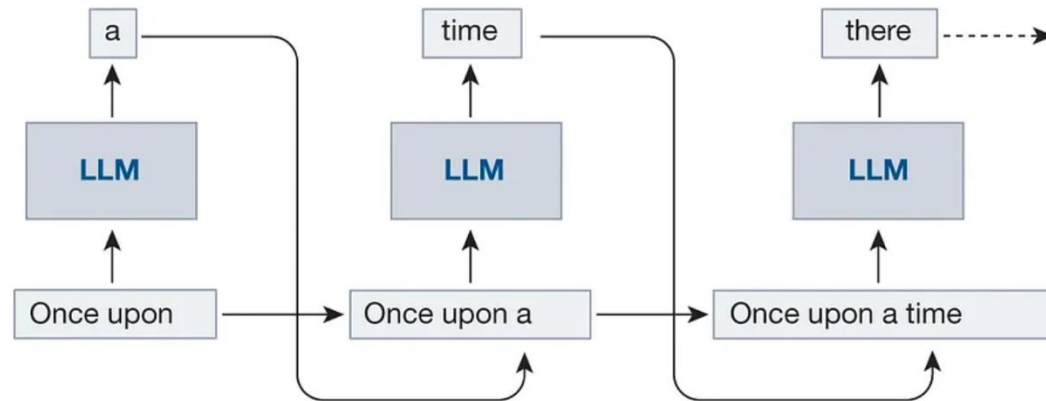


Fig 1. Autoregressive generation [1]

Text Generation

```
def autoregressive_decoding(x_seq, model, T):  
    n = len(x_seq[-1])  
    while n < T :  
        # get output probability distribution  
        q_next = get_probabilities(x_seq, model)  
        # sample next output token  
        x_next = sample(q_next)  
        # append next token to the input sequence  
        x_seq = torch.cat([x_seq, x_next], dim=-1)  
        n += 1  
  
    return x_seq
```



```
from transformers import AutoTokenizer, GPT2LMHeadModel  
import torch  
  
tokenizer = AutoTokenizer.from_pretrained("openai-  
community/gpt2")  
model = GPT2LMHeadModel.from_pretrained("openai-  
community/gpt2")  
  
text = "Once upon a time"  
input_ids = tokenizer.encode(text, return_tensors='pt')  
  
T = 10  
outputs = model.generate(input_ids, max_new_tokens = T)  
  
generated_text = tokenizer.decode(outputs[0],  
skip_special_tokens=True)  
  
print(f"Generated text: {generated_text}")
```

Greedy decoding (Deterministic Methods)

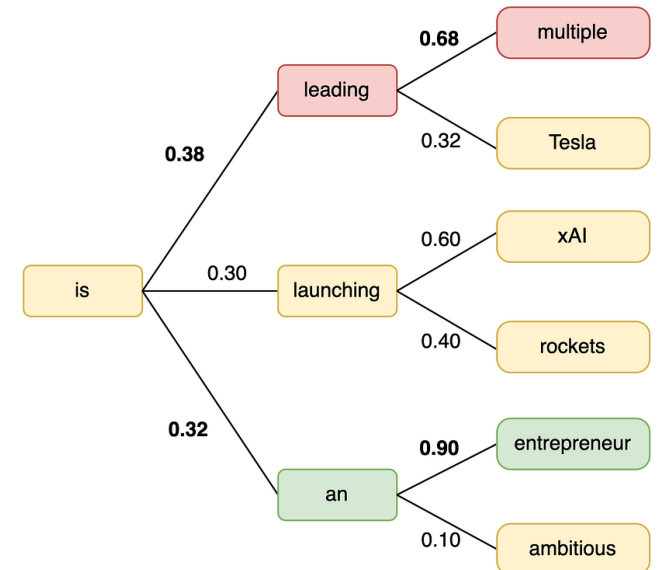
```
def autoregressive_decoding(x_seq, model, T):  
    n = len(x_seq[-1])  
    while n < T :  
        # get output probability distribution  
        q_next = get_probabilities(x_seq, model)  
        # sample next output token  
        x_next = sample(q_next)  
        # append next token to the input sequence  
        x_seq = torch.cat([x_seq, x_next], dim=-1)  
        n += 1  
  
    return x_seq
```

Pick the highest-prob token each step

- Fast, deterministic ✓
- Can be repetitive / get stuck in dull outputs ✗

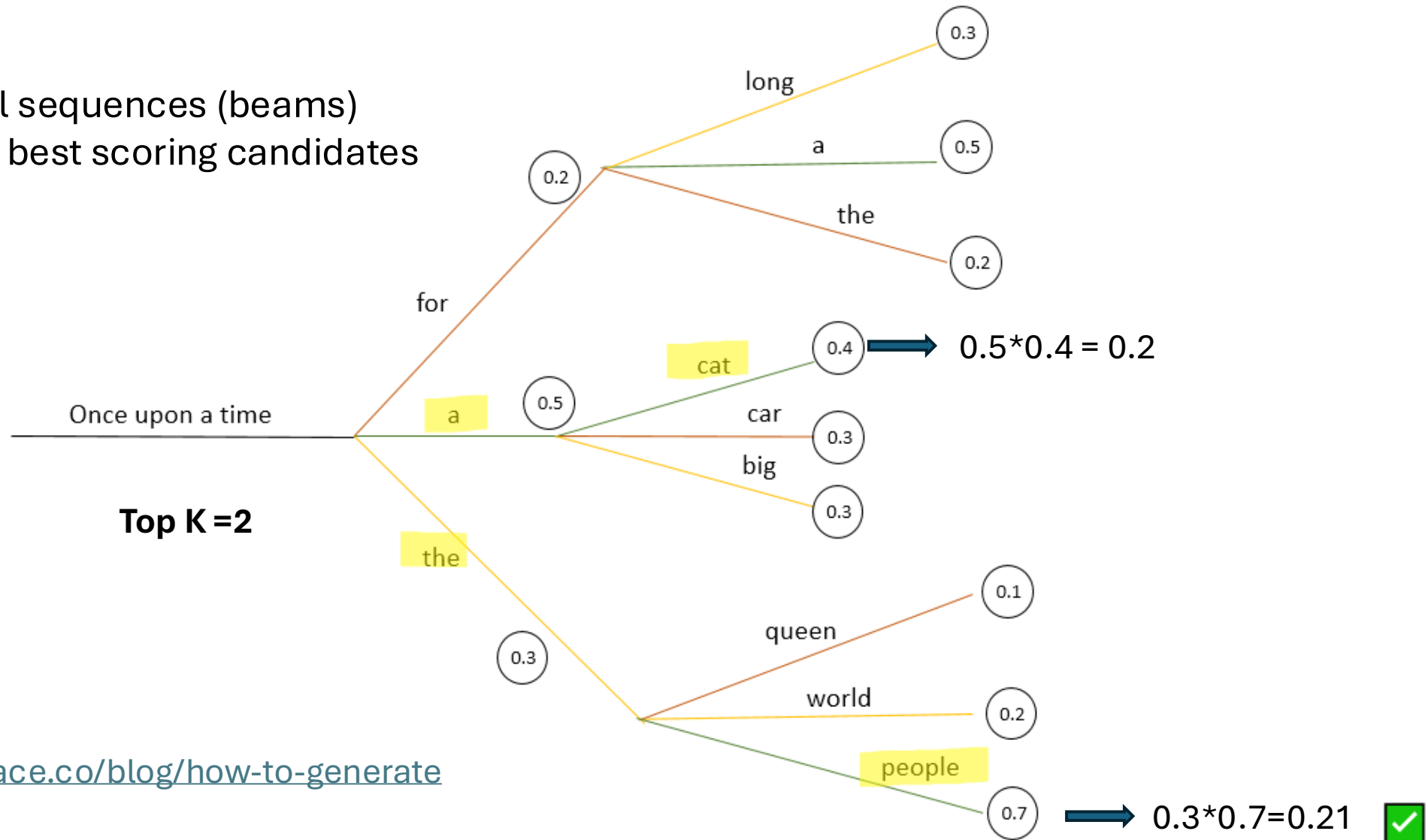
```
def sample(p):  
    return np.argmax(p)
```

Elon Musk



Beam Search

- Keep top **B** partial sequences (beams)
- Expand and keep best scoring candidates



Beam Search

Good for translation / structured tasks

- Those tasks often have a relatively “correct” answer.
- Beam search is good at finding a **high-probability** (most likely) full sentence.

Weak for chat

- In open-ended chat there are many valid answers.
- Beam search tends to pick the **most “safe/common” high-probability** phrasing, which often sounds:
 - generic (“It depends...”, “As an AI...”)
 - repetitive
 - less creative or less specific

Sampling

greedy search → terse, repetitive text

beam search → infeasible due to its high latency.

Goal: increase the creativity of our generations without sacrificing latency

Non-deterministically sample the tokens to generate from the output distribution of the LLM.

Step 1:

Convert the LLM's output
Into probability distribution

```
probs = F.softmax(o) # Convert o, the logits, into probabilities  
token = torch.multinomial(probs, 1) # sample one token from  
the distribution
```

Sampling

Step 1:

Convert the LLM's output
Into probability distribution

```
probs = F.softmax(o) # Convert o, the logits, into probabilities  
token = torch.multinomial(probs, 1) # sample one token from  
the distribution
```

Sampling could mean that tokens with extremely **low probability get generated!**

To reduce the → modulate
the *temperature* of the softmax operation.

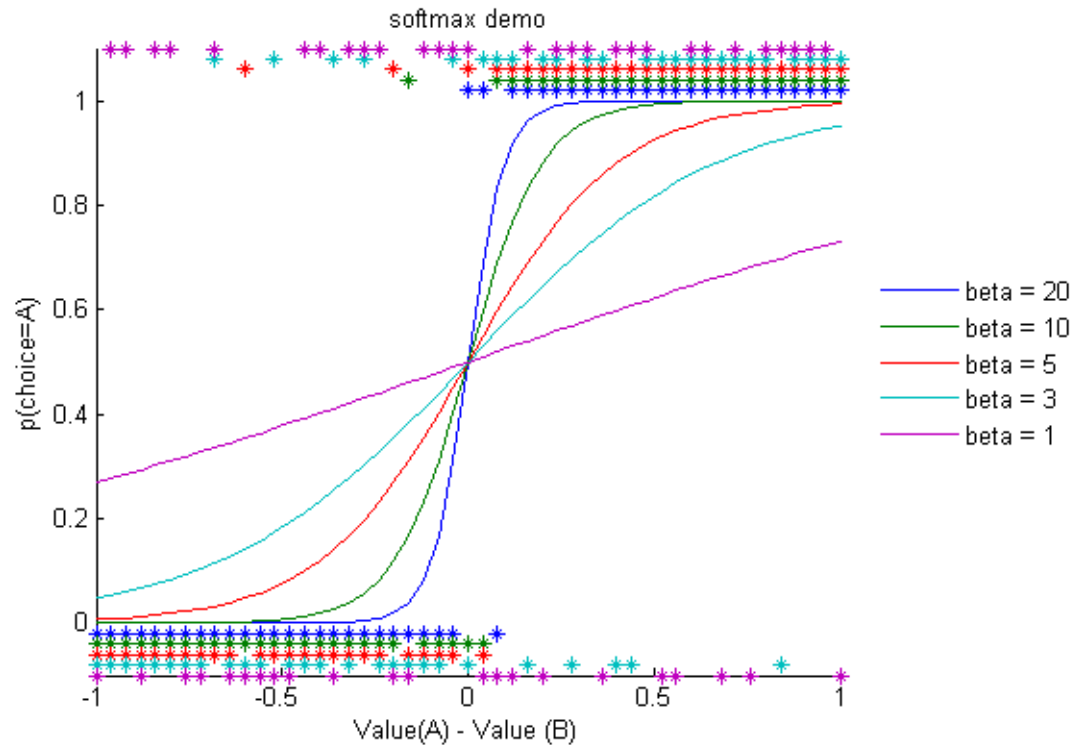
$$p_i = \frac{\exp(l_i/T)}{\sum_j \exp(l_j/T)}$$

T=1 ?

Lower T ?

Higher T ?

Sampling



`F.softmax(o)` # Convert o, the logits, into probabilities
`torch.multinomial(probs, 1)` # sample one token from
ribution

extremely **low probability** get generated!

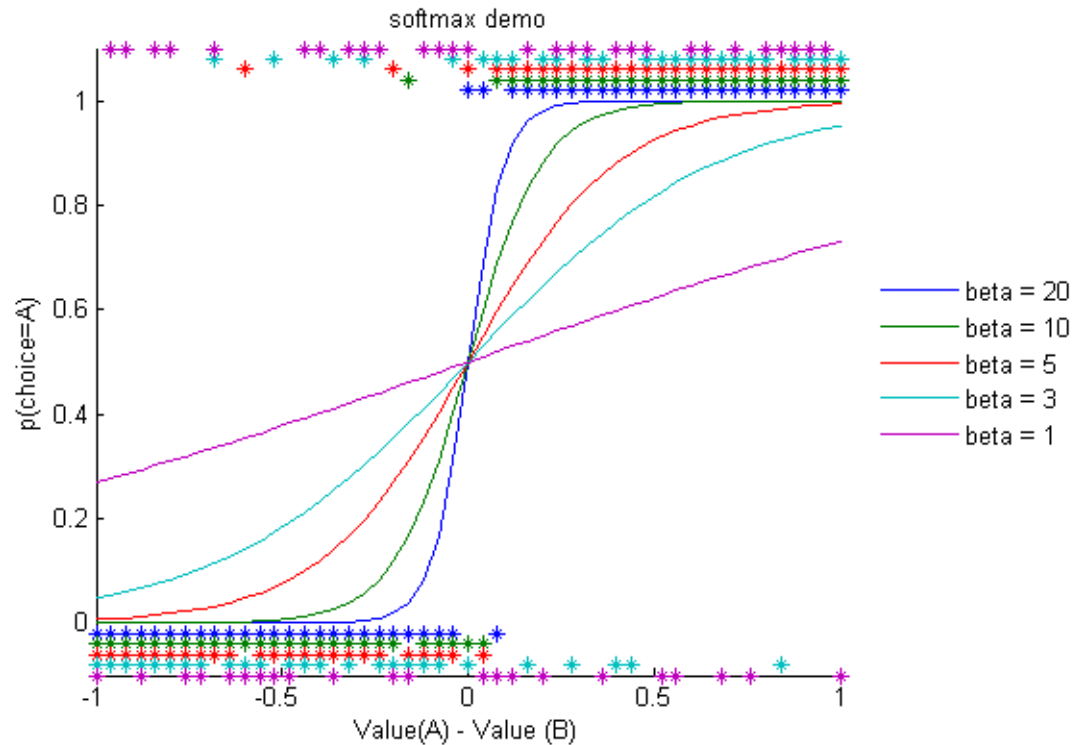
$$p_i = \frac{\exp(l_i/T)}{\sum_j \exp(l_j/T)}$$

T=1 base softmax

Lower T ? Beta ↑ sharper probability distribution

Higher T ? Beta ↓ diffuse distribution

Sampling



```
F.softmax(o) # Convert o, the logits, into probabilities  
torch.multinomial(probs, 1) # sample one token from  
ribution
```

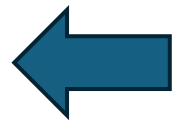
extremely **low probability** get generated!

$$p_i = \frac{\exp(l_i/T)}{\sum_j \exp(l_j/T)}$$

T close to zero

→ give more deterministic
outputs

Higher → More creativity



T=1 base softmax

Lower T Beta ↑ sharper probability distribution

Higher T Beta ↓ diffuse distribution

Sampling

But don't want the LLM to sample from the *entire* vocabulary

Motivation: Introduce some **randomness**, but don't really want to generate outlandish tokens.

Randomly sample the final token from **fixed size set**

→ **Top -K sampling**

Sampling

But don't want the LLM to sample from the *entire* vocabulary

Motivation: Introduce some **randomness**, but don't really want to generate outlandish tokens.

Randomly sample the final token from **fixed size set**

→ **Top –K sampling**

However

- if the set size k was **too small**, the responses would be **repetitive and boring**,
- if **too large**, → lead to the **generation of outlandish tokens** from the ***tail of the probability distribution***.

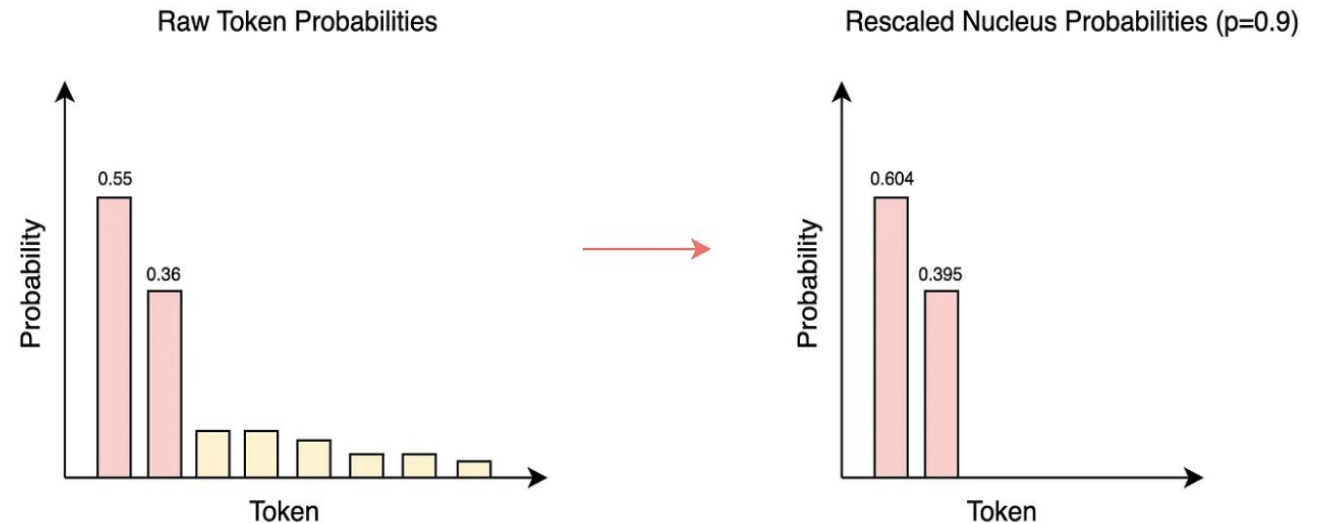


Top – p sampling

Nucleus Sampling

sample from smallest set with cumulative **prob $\geq p$**

- Choose the p value to use as the **threshold**. ($p=0.9$ in the example)
- Generate the **output probability** distribution over the whole vocabulary using a **softmax with some fixed temperature**.
- **Sort** the vocabulary by probability.
- Select the **minimum set** of tokens whose probabilities summed are **greater than p** .
- Re-scale the probabilities
- Sample from this new distribution.



Sampling

- **Top-k**: fixed number of candidates
- **Top-p**: variable number, depends on distribution “sharpness”

Overall

- **Factual/precise**: greedy or low-temp sampling
- **Creative**: sampling + higher temp
- **Avoid repetition**: repetition penalty / no-repeat ngrams
- **Beam search**: when you need globally high-prob sequences (classic NLP)

However, decoding is the bottleneck is not solved

Speculative decoding

- **Compute** (one forward pass per token)

Speculative decoding

Observation 1: Some tokens are easier to generate than others

What is the square root of 7? The square root of **7** is **2.646**.

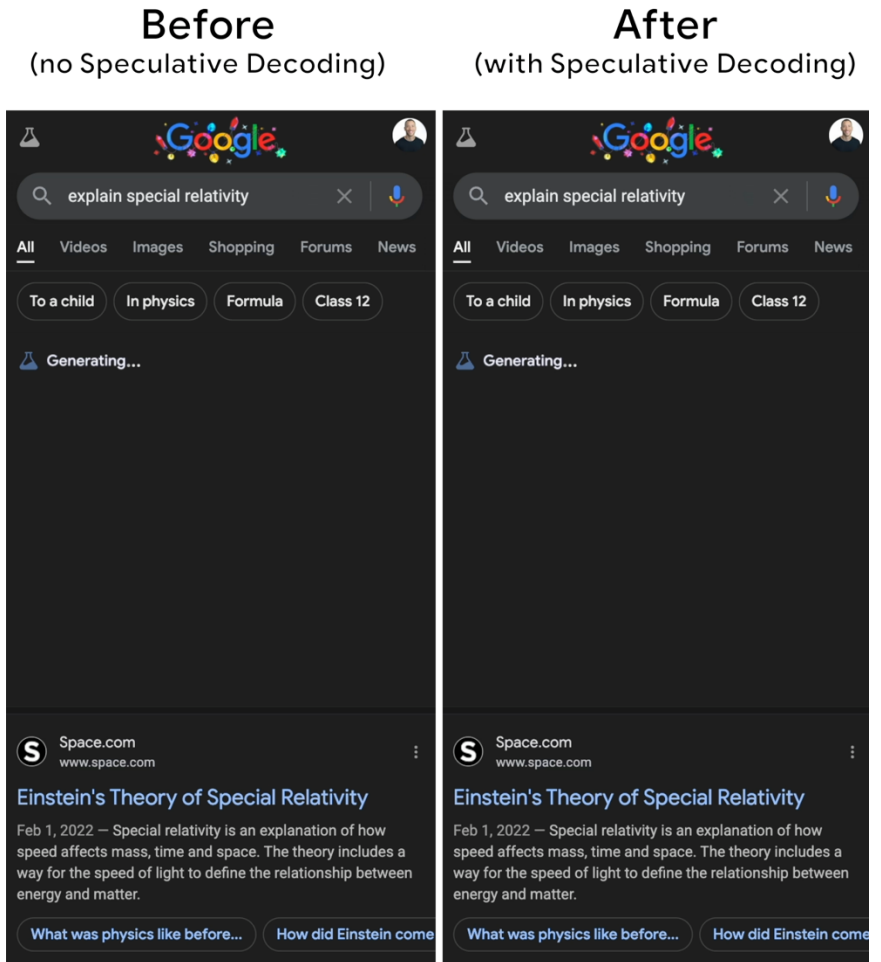
easier

harder

Observation 2: The bottleneck for LLM inference is usually memory

Hardware can do	Transformers need
~100s-1000s operations/byte read	~10 operations/byte read

Moving data in/out of memory is much slower
→ Hardware need to wait memory transfer



<https://research.google/blog/looking-back-at-speculative-decoding/>

Speculative decoding

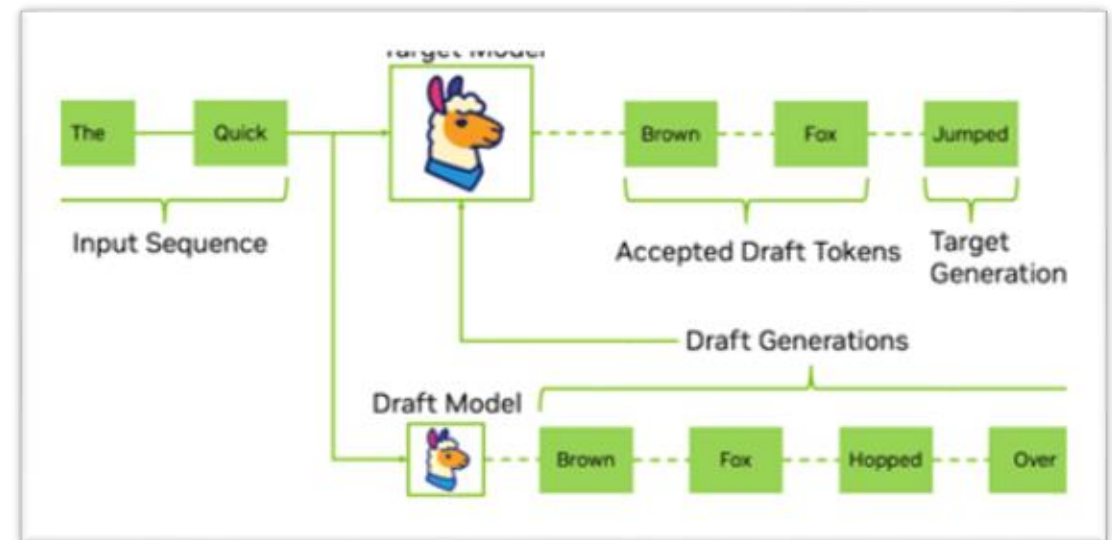
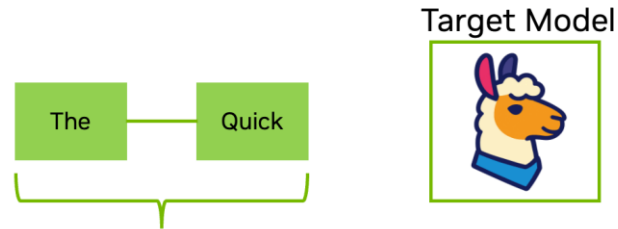
- Use a **small draft model** to propose multiple tokens (3-12 tokens)
→ small model trained on **same data distribution**
- Large model **verifies** instead of generating every token
- If many tokens are accepted → **fewer big-model forward passes**

[Accelerating Large Language Model Decoding with Speculative Sampling](#), Deep Mind

[Fast Inference from Transformers via Speculative Decoding](#)

Google research

<https://developer.nvidia.com/blog/an-introduction-to-speculative-decoding-for-reducing-latency-in-ai-inference/>



Speculative decoding- Parallel Verification (Why it's Fast)

- **Target (large) model verifies in parallel:** processes the **prefix + all draft tokens in one forward pass**, producing probabilities for each position.
- **Key efficiency gain:** replaces many sequential decode steps with **one batched verification step**, keeping the GPU busy.
- **KV cache reuse:** prefix KV is already cached → verification mainly pays cost for the **new speculated tokens**.
- **Accept/reject:** keep the **longest accepted prefix** of draft tokens → this becomes the **new prefix** for the next step.

The draft model proposes next 3 tokens based on context

The weather is quite → nice today outside

The target model verifies proposed tokens in parallel and accepts longest agreeable prefix

The	weather	is	quite	→	nice		✓
The	weather	is	quite	→	nice	today	✓
The	weather	is	quite	→	nice	today	outside ✗

The target model accepts 2 tokens and generates 1 new token to continue sequence

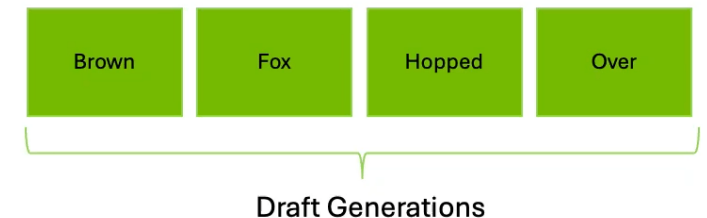
The weather is quite → nice today for

Speculative decoding- Reject sampling

Reject? Accept?

- If the **target model's predicted probability $q(x)$** is **higher** than the **draft model's $p(x)$** , **keep** the token.
- If not, then **with probability $1 - q(x)/p(x)$** , **replace the token** with a token sampled from $\max(0, q(x) - p(x))$.

- If token is **accepted**, it becomes part of the output.
- When you hit the **first rejected token**, you stop accepting further draft tokens.
- Then you generate the next token directly from the **target model** (and continue with a new draft block).



Speculative decoding

Helps most when:

- Draft model is much faster
- Large model verification is efficient
- Output is predictable enough that many tokens get accepted

Doesn't help much when:

- Draft tokens are often rejected (very creative tasks, high temperature)
- Draft model is too weak / too slow

Speculative decoding

Helps most when:

- Draft model is much faster
- Large model verification is efficient
- Output is predictable enough that many tokens get accepted

Doesn't help much when:

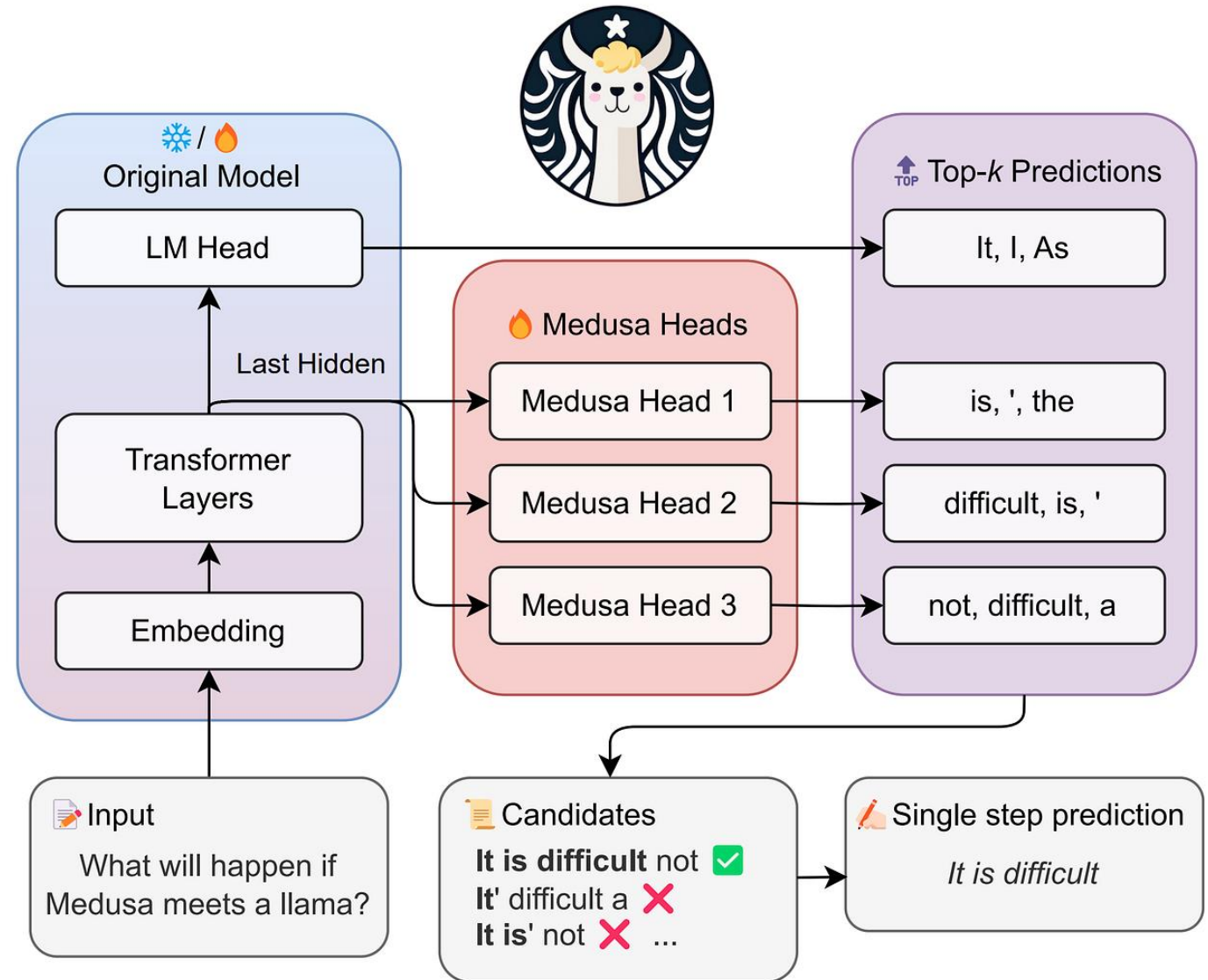
- Draft tokens are often rejected (very creative tasks, high temperature)
- Draft model is too weak / too slow

However...

The need for a separate draft model is too **cumbersome**. Can we do away with it?

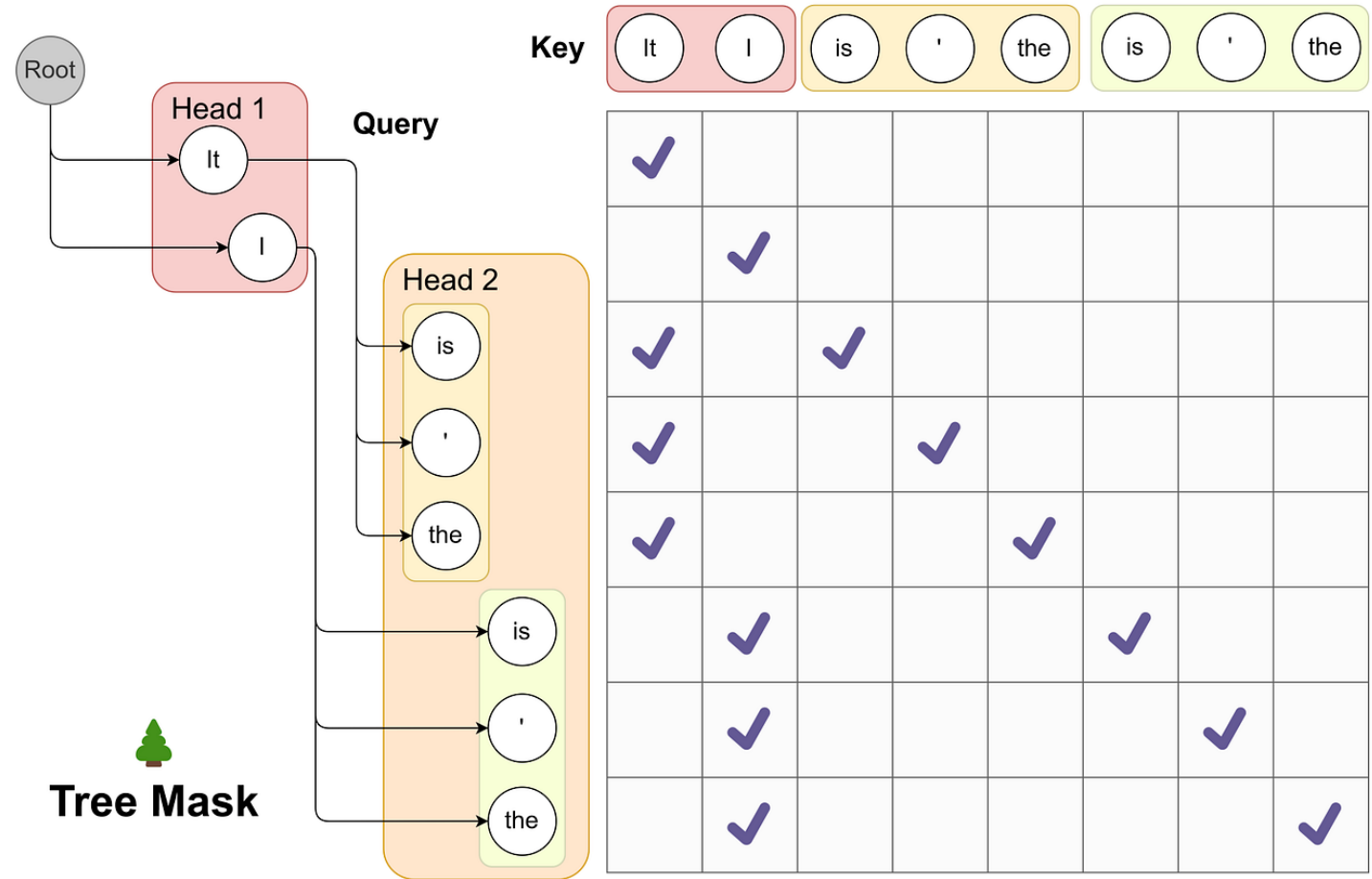
MEDUSA

- Instead of a separate draft model, you add **multiple “heads”** on the *same model* to propose **several future tokens** (a token “tree”) in one go.
- Then you **verify/accept** along a path.



MEDUSA

- **Add k additional LM heads to the transformer.** Specifically, we take the last **hidden state of the model**. Each LM head is a simple **2-layer FFN with a SiLU activation and residual connection** in the middle.
- **Each of the LM heads is finetuned.**
- **Keep track of the top s_k tokens generated by each head**
- **Tree attention**, to verify the entire tree of candidates at the same time.

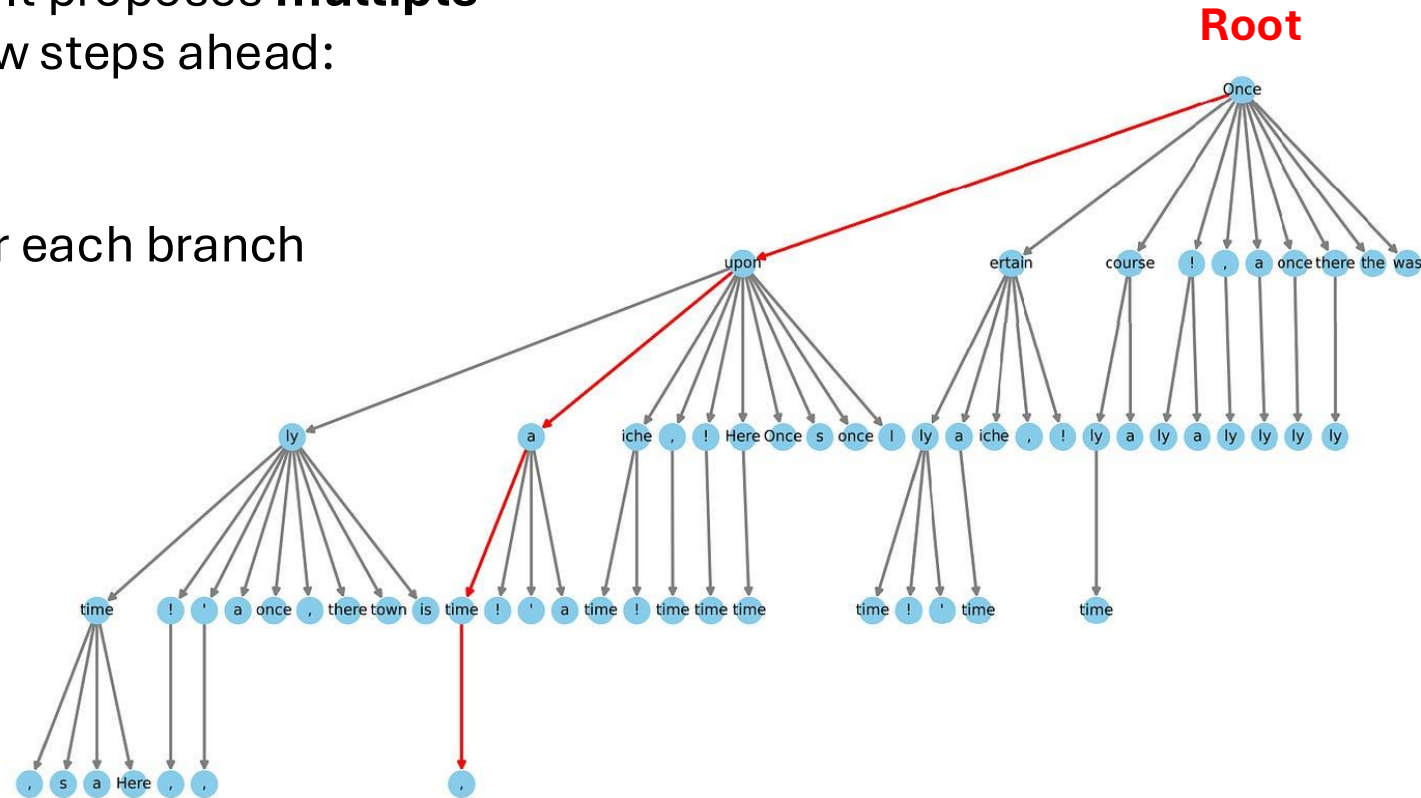


MEDUSA- Tree Attention

Not just propose *one* next-token sequence. It proposes **multiple possible continuations** (branches) for a few steps ahead:

- root = your current prefix
- level 1 = several candidate next tokens
- level 2 = several candidate 2nd tokens for each branch
- ... forming a **token tree**

- **Shares computation for the common prefix**
 - Stores **KV cache for the tree nodes**.
- When evaluating a node, it attends to:
- the **original prefix tokens**, plus
 - the **node's ancestors** in the tree (the path from root $\rightarrow$ that node)



MEDUSA



Next steps

- Evaluation metrics: semantic evaluation, human evaluation and system-level evaluation

Paper task: **EAGLE: Speculative Sampling Requires Rethinking**

Feature Uncertainty

Additional Material:

1. <https://www.kaggle.com/whitepaper-foundational-llm-and-text-generation>
2. [Accelerating Large Language Model Decoding with Speculative Sampling](#), Deep Mind
3. [Fast Inference from Transformers via Speculative Decoding](#)
Google research

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?

